

PART VII — PRODUCTION, DEPLOYMENT, AND BEYOND

Chapter 39

Performance and Cost Optimization

“A pipeline that is correct but slow is not yet finished; it is finished when the seconds and the dollars have been accounted for as carefully as the answer was.”

Chapter 39 — Performance and Cost Optimization

Chapter 23 already established the token-budget math that determines what fits in a context window; this chapter turns to the second constraint every production system eventually has to answer for — what the pipeline costs in wall-clock seconds and in dollars, and which of those costs are worth spending engineering effort to reduce. This project's own `timing_tracker.py` singleton has been recording exactly this data on every run since ADR-043, which makes this chapter unusually well grounded: the long-tail latency numbers, the timeout values, and the concurrency-control design this chapter covers are all measured facts from real runs, not estimates.

It is equally important to be honest about what this project never built. There is no embeddings cache, no retrieval cache, no LLM-response cache, and no index quantization anywhere in this codebase — this chapter says so plainly where it applies, rather than describing tooling that does not exist as though it does. What the project does have — calibrated per-call timeouts, a fairness-preserving concurrency gate, a three-way LLM cost/capability split, and a very literal illustration of why call-count reduction (Chapter 36C) was the biggest lever this project ever pulled — is real, measured, and worth teaching in detail.

39.1 Batching embeddings — sweet-spot batch sizes

This project's own GPU-ingestion research found the sweet spot empirically rather than by formula: a batch size of 128-512 suited an RTX 5050 laptop GPU's 8GB of VRAM running `all-MiniLM-L6-v2`, with diminishing returns observed beyond 256. The live default, `BATCH_SIZE = 512` in `app_workflow/config.py`, sits at the top of that observed range. Worth noting for anyone tempted to blame batch size for a slow ingest: the original slowness this research was chasing turned out not to be a GPU-throughput problem at all — it was caused by writing to ChromaDB once per small batch rather than embedding in bulk and inserting in bulk, and the real fix was restructuring the write pattern, not tuning the embedding batch size in isolation.

39.2 Caching at every layer

Common pitfall — Assuming a self-learning RAG system has a cache layer

This project has no embeddings cache, no retrieval cache, and no LLM-response cache anywhere in its codebase — an exhaustive search for `cache`, `lru_cache`, and similar patterns across both pipelines confirms this. The only real functional analog is `SelfLearner`'s distillation into a `learned_qa` collection (Chapter 32): every five successful interactions get distilled and re-embedded, and a future similar query can retrieve that distilled answer directly — which behaves like a semantic response cache in effect, even though this project's own documentation never frames it that way.

The general literature's three standard caching layers are each worth understanding even though this project implements none of them directly. An embeddings cache keyed on exact text avoids re-encoding identical chunks across repeated ingestion runs. A retrieval cache keyed on a normalized query (Chapter 30's `_normalize_query()` pattern would be the natural key) avoids repeating an expensive vector search for a query variant this project has already seen. An LLM-response cache keyed on the full prompt avoids paying for an identical generation twice — valuable specifically for the validator prompts Chapter 37 covered, which are far more likely to see near-identical inputs across requests than final-answer generation is.

The reason this project never built any of the three is worth stating rather than glossing over: its actual traffic pattern — a small number of users, genuinely varied queries, and a self-learning layer already designed to surface repeat-relevant content through retrieval rather than exact-match lookup — never produced the cache-hit-rate justification a formal cache layer needs to earn its complexity. A reader

building a higher-traffic system should treat this section as a checklist of what to add, not as a description of what this project already has.

39.3 Reducing token usage in prompts

Chapter 22B's semantic compression is this project's real, measured answer to prompt token reduction — LBC achieved roughly a 27.6% size reduction in testing, trading a small amount of LLM call time for a meaningfully smaller downstream prompt. A second, structural technique appears at the conversation-history level rather than the retrieved-context level: once a request's token count crosses a 500-token gate, the compression pipeline (NAC, DC, LBC) is skipped entirely rather than run and then discarded, and prior raw `retrieve_documents` tool-result messages are scrubbed from the conversation history and replaced with a placeholder specifically to keep the context window lean across a multi-turn agent loop, rather than letting every prior retrieval's full text accumulate turn over turn.

39.4 Picking cheaper models for cheap steps

Section 39.11 covers this project's real three-way LLM split in detail; the principle behind it is simple enough to state on its own: a step that only needs to emit a binary or small-enum verdict — PASS/FAIL, GROUNDED/OVERCLAIMED/etc. — does not need the same model capacity as a step that has to synthesize a coherent, well-reasoned final answer from scratch. Routing the first kind of step to a smaller, cheaper model and reserving the largest model for genuine generation is the same idea Chapter 25 covered from the small-model-capability angle, applied here specifically as a cost lever rather than a correctness one.

39.5 Async and parallel retrieval

This project's own cross-pipeline benchmark is the clearest possible demonstration of parallel retrieval's real cost impact: on the same query set, the LangGraph pipeline (`app_workflow`, using LangGraph's `Send()` fan-out to dispatch every query variant concurrently) averaged 13 minutes 51 seconds total with 5 minutes 16 seconds spent on retrieval, against the older sequential `app` pipeline's 28 minutes 28 seconds total with 18 minutes 16 seconds on retrieval — roughly a 2x overall speedup, concentrated almost entirely in the retrieval stage. `dedup_merge.py`'s own inline comment states the design directly: the per-track dedup-merge nodes for the documents and learned-QA collections "run in parallel," reflecting the same two-track design Chapter 33 introduced from a retrieval-correctness angle now paying off as a latency win too.

39.6 Index compression and quantization

Common pitfall — Assuming a project using ChromaDB has quantized its index

This project's vector store uses ChromaDB's default HNSW index, uncompressed — no product quantization, scalar quantization, or IVF clustering appears anywhere in the codebase or research ledger. This project's own research explicitly frames HNSW as future-proofing for a corpus that, at roughly 1,181 chunks, was still small enough that exact brute-force search remained feasible; HNSW's approximate ~95-99% recall was adopted for headroom, not because compression was already a bottleneck.

For a corpus large enough that HNSW's memory footprint itself becomes the constraint — not this project's scale, but a plausible next stage for a reader's own system — product quantization compresses each vector into a small set of learned sub-vector codes, trading a controlled amount of recall accuracy for a large reduction in index memory. This is genuinely external-literature material relative to this project's own history: worth knowing, not worth presenting as something this codebase has done.

39.7 Thread-based per-call timeouts

Definition — `ThreadPoolExecutor.submit(...)` + `future.result(timeout=N)`

A portable pattern for imposing a hard wall-clock timeout on any blocking call: submit the call to a single-worker thread pool, then wait on the returned future with an explicit timeout, raising a `concurrent.futures.TimeoutError` if the call has not completed in time.

ADR-042 and this project's own research record the reasoning behind choosing this pattern over the alternatives, and both rejected alternatives are worth knowing specifically because they are the first thing most engineers reach for. `signal.alarm` is Windows-incompatible and cannot interrupt a call that is blocked inside a C extension holding the GIL — a real constraint for a project whose embedding and LLM libraries do exactly that. Per-library timeout parameters were the other rejected option, simply because neither ChromaDB nor SentenceTransformers exposes one for the specific calls this project makes. The thread-pool pattern works identically across platforms and around any library that offers no timeout hook of its own:

```
with concurrent.futures.ThreadPoolExecutor(max_workers=1) as _executor:
    _future = _executor.submit(llm.invoke, messages, **kwargs)
    try:
        response = _future.result(timeout=LLM_RESPONSE_TIMEOUT_SECONDS)
    except concurrent.futures.TimeoutError:
        raise LLMResponseTimeoutError(LLM_RESPONSE_TIMEOUT_SECONDS)
```

This exact pattern appears at six separate call sites across both pipelines — `llm_caller.py`, `retriever.py`, and `embedding_manager.py`, each with a timeout value BUG-051 calibrated from real measured p95 latencies rather than a guessed round number: `LLM_RESPONSE_TIMEOUT_SECONDS = 150` against a measured LLM p95 of roughly 75 seconds (2x headroom), `RETRIEVAL_TIMEOUT_SECONDS = 10` against a measured ChromaDB p95 of roughly 0.7 seconds (about 14x headroom), and `EMBEDDING_ENCODING_TIMEOUT_SECONDS = 5` against a measured encode p95 of roughly 0.02 seconds (about 250x headroom). Figure 39.1 shows the pattern's one real limitation.

The portable per-call timeout — and its one real caveat

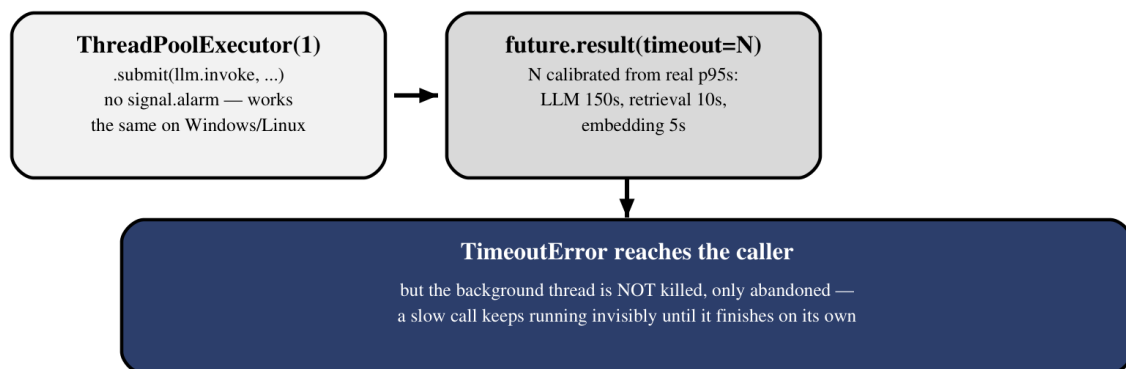


Figure 39.1 — `future.result(timeout=N)` reliably stops the caller from waiting, but the abandoned thread keeps running invisibly to completion; this pattern bounds wait time, not resource usage.

The limitation Figure 39.1 names is stated explicitly in this project's own code comments and worth repeating verbatim in spirit: the background thread is not killed when a timeout fires, only abandoned. A

caller stops waiting for the result, but the slow call itself keeps consuming CPU, holding a connection, or waiting on a hung external service until it eventually finishes (or the process exits) — this pattern bounds how long a caller waits, not how much work the abandoned call continues to do in the background.

39.8 Semaphores vs FIFO queue for LLM-call serialization

BUG-053 is the concrete failure that forced this project to move past a naive concurrency primitive: under two genuinely concurrent `/query` requests, an `asyncio.Lock` failed to serialize Groq API calls correctly, because `asyncio.Lock` is not a cross-thread mutex and this project's calls were crossing thread boundaries (Section 39.7's thread-pool timeout pattern is exactly the kind of boundary that breaks naive `asyncio` synchronization). This project's own research is explicit about why a plain counting semaphore would not have been the right replacement either: "Semaphores are insufficient for the token-quota exhaustion problem because they provide no guarantee that a 429-holding thread will regain the gate before new arrivals" — a semaphore enforces a concurrency limit, but says nothing about serving order, so a thread that just hit a rate limit could be starved indefinitely by newly arriving requests cutting in front of it.

ADR-044's replacement is a genuine FIFO queue built from `queue.Queue[threading.Event]` combined with a `threading.Lock` — `_gate_acquire()` and `_gate_release_to_next()` in `llm_caller.py` — guaranteeing that whichever thread has been waiting longest is the next one released, regardless of how many new requests arrive while it waits. The distinction this bug and its fix teach generalizes past this specific project: a semaphore answers "how many can run at once," while a FIFO queue additionally answers "in what order do waiters get served," and a system where fairness under rate-limit pressure matters needs the second guarantee, not just the first.

39.9 Exponential backoff with jitter as an architectural pattern

Common pitfall — Assuming this project's retry logic still uses jitter

`llm_caller.py`'s `_rate_limit_delay()` still computes classic exponential backoff (`base_seconds * 2**(attempt - 1)`), but jitter was deliberately removed. Its own docstring explains why: "Jitter is intentionally absent: with the FIFO gate only one thread calls Groq at a time, so there is no collision to de-synchronize."

This removal is itself the more interesting lesson than the backoff formula alone. Jitter — randomizing the exact delay so that multiple clients backing off from the same failure don't all retry at the exact same instant and collide again — solves a **concurrent-collision** problem specifically. Once Section 39.8's FIFO gate guaranteed that only one thread ever calls the LLM provider at a time, the collision jitter exists to prevent became structurally impossible, and the parameter that removed it (`LLM_RATE_LIMIT_BACKOFF_JITTER_SECONDS`) was deleted from `config.py` entirely rather than left in place unused. The architectural lesson: exponential backoff and jitter solve two different problems — pacing retries, and desynchronizing concurrent retriers — and a concurrency-model change elsewhere in a system can make the second one moot even while the first remains necessary.

39.10 The GPU-driver failure fallback

BUG-057, still open in this project's bug ledger, documents a specifically Windows hardware failure: an RTX 5050 laptop GPU reporting `ConfigManagerErrorCode = 43` — a driver-level crash state — causing `torch.cuda.is_available()` to return `False` even though the installed `torch==2.11.0+cu128` build is genuinely CUDA-capable, and `nvidia-smi` itself failing with exit code 4. The bug record is explicit that this is a Windows driver failure, not application code: "Not a PyTorch or project-code issue."

The fallback that keeps the pipeline running through this failure is a single line in `embedding_manager.py`: `self.device = "cuda" if torch.cuda.is_available() else "cpu"`. There is no explicit Code-43 detection anywhere in the code — the fallback is unconditional and automatic, silently routing to CPU embeddings whenever CUDA reports itself unavailable for any reason at all, Code 43 included. This is a deliberately conservative design: rather than trying to distinguish a genuine driver crash from a machine that simply has no GPU, the code treats "CUDA unavailable" as a single condition with a single fallback, accepting slower CPU-bound embedding generation over a hard failure of the entire ingestion or query path.

39.11 The merge_llm / judge_llm / json_fix_llm split

ADR-018 introduced this project's original three-instance model split, and its own rationale is worth quoting directly: "`judge_llm` can be upgraded to `llama-3.3-70b-versatile` ... while `llm` stays on `llama-3.1-8b-instant`" — the explicit design intent was decoupling the model powering user-facing generation from the model powering internal judgment calls, so each could be tuned independently for its own cost/capability tradeoff. The live configuration keeps `judge_llm` on `Qwen2.5-7B-Instruct` at temperature 0.0 (deterministic, appropriate for a verdict-producing judge) and adds a fourth, even smaller model — `json_fix_llm`, on `Qwen2.5-Coder-3B-Instruct` — dedicated purely to structural JSON repair rather than reasoning, per ADR-055.

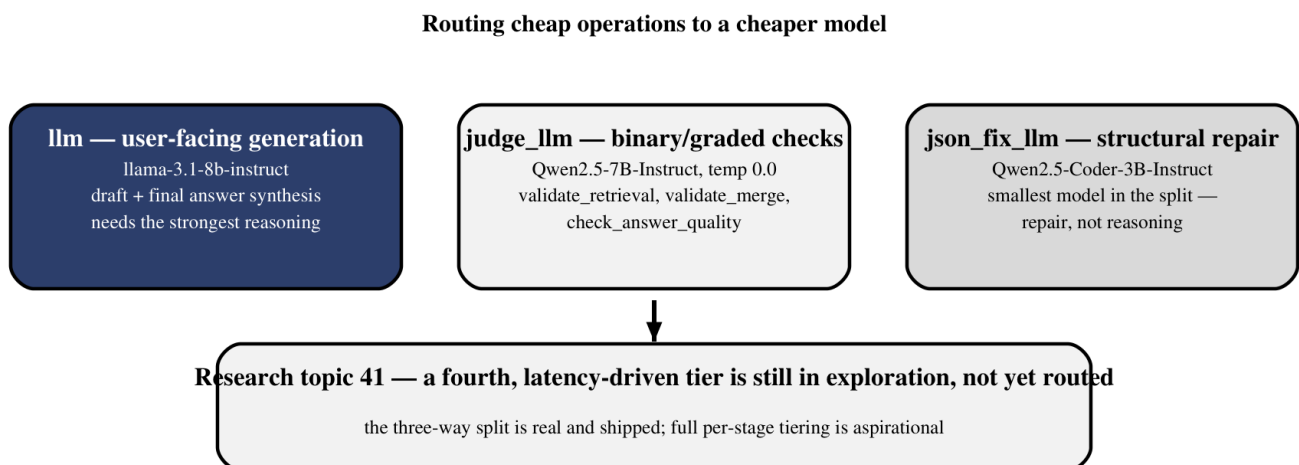


Figure 39.2 — Three models, three cost/capability points: the smallest model does the narrowest job, and only the user-facing generation step pays for the largest one.

Figure 39.2 is worth reading against Research topic 41's proposed further extension, which classifies pipeline stages into three latency/cost tiers — binary-judge steps (a 3B-class model is sufficient), faithful-rewrite steps (needing roughly 4-7B capability), and user-facing generation (needing the strongest available model) — but the record is explicit that this classification remains in an exploration phase, with no code changes committed toward full per-stage routing. What is genuinely live in production is the three/four-instance split this section describes in detail; the fully generalized tiered-routing vision is a real, sourced research direction this project has not yet built.

39.12 Latency budgeting in a multi-stage pipeline

The most important finding in this project's own latency data is where the seconds concentrate: BUG-051's calibration numbers put LLM call p95 latency at roughly 75 seconds against a ChromaDB retrieval p95 of roughly 0.7 seconds and an embedding-encode p95 of roughly 0.02 seconds — a difference of two to four orders of magnitude. Nearly every second a request spends in this pipeline is spent waiting on an LLM call, not on retrieval or embedding, which is precisely why Chapter 36C's threshold-tuning work — reducing LLM call **count** from 148 to 30 on a simple query and from 205 to 30 on a complex one — was a far larger latency win than any retrieval-side optimization this chapter covers could have been on its own.

This is the chapter's central lesson stated plainly: when nearly all latency lives in one stage type, the highest-leverage optimization is reducing how many times that stage type gets invoked, not making each invocation marginally faster. Section 39.5's parallel-retrieval win and this section's call-count observation are not competing explanations for this project's real 2x cross-pipeline speedup — they are complementary, but the call-count reduction from Chapter 36C's evidence-based tuning did more of the actual work, because it attacked the stage where the seconds were actually concentrated.

39.13 The singleton `timing_tracker.py`

``app/timing_tracker.py`` is a small, deliberately simple singleton module — its own docstring states the design directly: "Singleton timing tracker that records per-phase durations to a JSON file. Initialized by ``logger_config.setup_logging()``; all other modules call ``record()`` or ``record_llm()`` without needing the file path." Four functions cover the entire interface:

```
def initialize(json_path: Path) -> None: ...
def record(category: str, duration: float) -> None: ...
def record_llm(caller_tag: str, duration: float) -> None: ...
def _write() -> None: ...
```

``record_llm()`` resolves a caller tag like ``"AGENT-DRAFT"`` or ``"CAQ-JUDGE"`` to one of nine tracked categories — Sub-Query Generation, Total DB Retrieval Time, Total DB Retrieval Validation Time, Total Merge Time for Retrieved Chunks, Total Validation Time for Merged Chunks, Compression, Draft Generation, CAQ, and Final Generation — and no-ops silently for any tag it doesn't recognize, rather than raising. ``_write()`` flushes the entire accumulated JSON structure synchronously on every single call, a design ADR-043 defends explicitly against the performance cost it might seem to invite: "a run that crashes mid-way still produces a partially-populated JSON," trading a small per-call write cost for a durability guarantee that a hung or crashed run's partial timing data is never lost.

The resulting per-run JSON file is where this chapter's most striking number comes from, and it deserves to be quoted exactly rather than summarized: this project's own development record states that "the timing file includes, among other observations, retrieval-validation calls from milliseconds to roughly 102 s, merge calls up to roughly 310 s, merged-chunk validation up to roughly 374 s, and compression calls above 12 minutes." A long tail this wide — the same operation taking anywhere from single-digit milliseconds to over twelve minutes — is why ``timing_tracker.py`` records every individual call rather than only an aggregate mean per category: a mean across that distribution would hide the twelve-minute outliers entirely, and those outliers, not the typical case, are usually what a latency-budgeting effort actually needs to find and explain.